JESUN.AI FIELD BUILD LOG

# MUSE POTENTIAL EXTRACTION 101

A self-hosted n8n, connected to everything, for $0.

---

TUTORIAL AND SUPPORTING PACK

19 OCTOBER 2026

BY JESUN AHMAD USHNO

**TOTAL SPEND: $0**

THE BRAND OF RECEIPTS

## THE HOOK: THIS IS NOT A CHATBOT

Most people use an AI chat app like a smart search box. They ask a question. They get text back. Then they do all the real work themselves.

This tutorial shows a different way to use Muse.

Muse is not just a chatbot. It is a full sandbox. It is a powerful cloud computer in your pocket. It has its own machine. It has a terminal. It has a browser. It has files and memory. It runs scheduled jobs while you sleep. It can build real infrastructure, run it, break it, fix it, and report back to your phone.

In one night, from a phone, we used that machine to do a full infrastructure job:

- We ran a self-hosted automation server (n8n) on the cloud machine.
- We connected it to 5 Google services with one OAuth sign-in.
- We gave it a public address on a real domain.
- We put a login wall in front of it that lets exactly one person in.
- We did all of it on free plans. Total spend: $0.

This document is the full tutorial. Every step is real. Every number below was checked live during the build. The failures are in here too, with the fixes. That is the point of the series: receipts, not hype.

## WHAT WAS BUILT, AND WHY

The goal was simple. Jesun (the operator) wanted one automation hub that can reach his accounts and run work on a schedule. The hub is n8n, an open source workflow automation tool. Think of n8n as a control panel where you connect services with visual workflows, so tasks run on their own.

Why self-host n8n on the Muse machine instead of paying for a hosted plan?

- The machine already exists. Muse works on a Linux cloud computer (a sandbox VM) with a home folder that persists.
- Self-hosting costs nothing.
- The data stays on the machine. No third party holds the credentials store.
- The operator can watch every step from his phone and stop anything at any time.

At the start of the night, the machine already ran n8n version 2.43.5 as a local install, with a SQLite database and 8 starter workflows in the store. The machine had one big limit: no one could reach it from the internet. It had no public address. Fixing that limit, safely, is the heart of this tutorial.

## THE ARCHITECTURE IN PLAIN WORDS

Here is the final setup, piece by piece. Keep this picture in mind. Every step below builds one piece of it.

- **The operator.** Jesun. He gives orders from his phone (the iOS app) and checks results there. His home PC also plays a part.
- **Muse.** The agent. It lives in the chat and works on the cloud machine. It runs commands, edits files, tests results, and writes the report.
- **The sandbox cloud computer (the VM).** A Linux machine in the cloud. It runs n8n, the guard jobs, and a small helper loop. Its home folder survives machine restarts.
- **BlackBox (the home PC).** Jesun's Windows PC at home (Intel Core Ultra 7 255HX, 15 GB RAM, RTX 5050). It runs the tunnel connector program, cloudflared.
- **Tailscale.** A private network that links the VM and the PC. Only Jesun's own devices are on it.
- **Cloudflare.** It holds the domain DNS, runs the tunnel, and guards the front door with Zero Trust Access.
- **Google Cloud project jesun-n8n.** One project. Five APIs on. One OAuth credential family feeds all 5 Google services.
- **The public address.** https://n8n.jesunahmadushno.com, behind the Access wall.

The data path is short. A visitor opens the public address. Cloudflare checks the Access policy. If the visitor passes, the request rides the tunnel to the PC. The PC passes it through a private SSH forward to the VM. n8n answers. No port on any machine is open to the public internet.

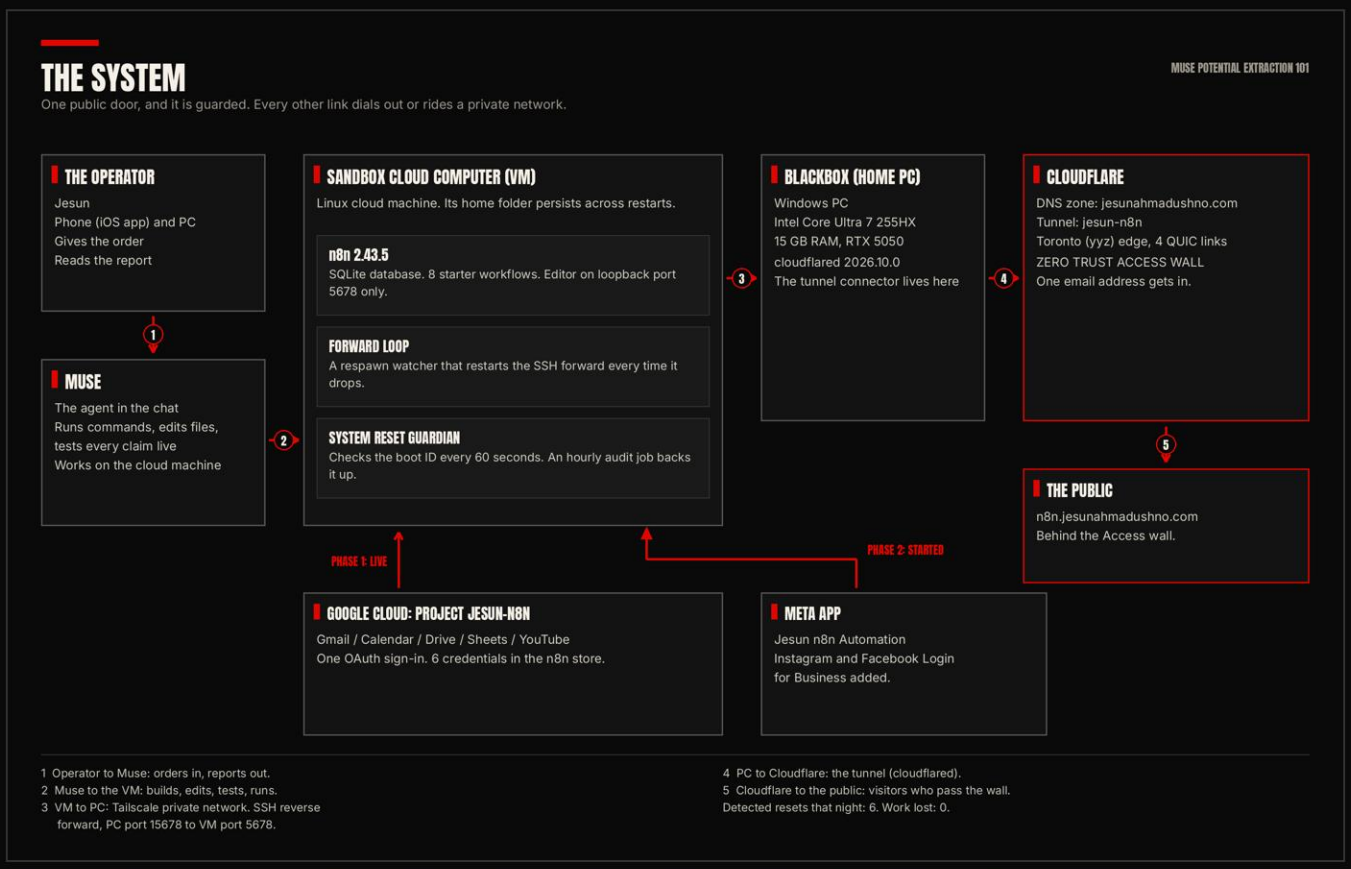



Figure 1. The full chain. A visitor passes the Access wall, rides the tunnel to the home PC, then crosses the SSH forward to n8n on the cloud machine.

# STEP BY STEP

## (A) SELF-HOST N8N ON THE SANDBOX MACHINE

n8n was already installed on the VM as a local install (not Docker). Key facts of this install:

- Version: 2.43.5.
- Database: SQLite, a single file on disk.
- Editor port on the machine itself: 5678. It answers only on the machine's own loopback address, 127.0.0.1. That means only software on the VM can open it. The public cannot.
- A start script launches it. A health address on the machine (the /healthz path) returns a simple OK status, so any script can check if n8n is alive.
- 8 starter workflows sat in the store at the start: content, inbox triage, ops snapshot, and pilot builds. They were all switched off. Nothing ran by accident.

Lesson: a loopback-only install is the safe default. The server works. The world cannot knock on it yet. You add the door later, on your terms.

## (B) CONNECT GOOGLE WITH ONE OAUTH CREDENTIAL (5 SERVICES)

Phase 1 was Google. One sign-in connected five services at once: Gmail, Google Calendar, Google Drive, Google Sheets, and YouTube.

The steps:

1. Create one Google Cloud project. The project here is named jesun-n8n. No billing account was created. Google Cloud works for this job without billing.
2. Turn on the 5 APIs in that project: Gmail, Calendar, Drive, Sheets, and the YouTube Data API v3.
3. Set up the OAuth consent screen. The app here is named "Jesun n8n". Set it to External and move it to "In production". This step matters. An app left in "Testing" status kills its refresh tokens after 7 days. "In production" avoids that trap.
4. Create one OAuth client (Web type). Register the redirect address that n8n gives you. In this build it used the localhost form of the n8n callback address, because step (a) has no public address yet. Google allows localhost redirects for this kind of client.
5. One trap appeared during consent: some Google scopes cannot be requested together. The first consent link failed with an error that named the clash (a narrow Drive scope mixed with YouTube scopes). The fix was simple: drop the narrow scopes that a wider scope already covers, and request again. The consent then passed.
6. Put the client ID and secret into n8n, complete the sign-in, and n8n stores the tokens in its encrypted credential store. Secret values never go into files, notes, or reports. Only the credential names and IDs are recorded.
7. One base credential came first. Then 5 typed copies were made from it, one per service, because each n8n service node wants its own credential type. The store then held 6 Google credentials: "Google Phase1 Jesun" plus Gmail, Calendar, Drive, Sheets, and YouTube copies, each marked "Phase1 Jesun".

Then came the tests. A connection is not real until a live call returns real data. All 5 services passed API-level tests with HTTP 200:

- Gmail returned real inbox subjects.
- Calendar returned real upcoming events.
- Drive listed real files.
- Sheets returned real rows from a real spreadsheet.
- YouTube returned the real channel, "Jesun Ahmad Ushno", with 69 subscribers at the time of the test.

Three services (Gmail, Calendar, YouTube) also passed a second test at node level, inside n8n workflows themselves. One honest gap stayed open: channel read on YouTube is verified. Upload is not tested yet. Do not claim uploads work until that test runs.

## (C) THE EXPOSURE PROBLEM AND THE 3 FAILED TUNNEL ROUTES

Now the hard part. A self-hosted server with no public address cannot receive visitors, webhooks, or OAuth callbacks from the public internet. The standard answer is a Cloudflare Tunnel: a small program (cloudflared) on your machine dials out to Cloudflare and holds the connection open. Visitors reach Cloudflare. Cloudflare sends their requests down the tunnel. No inbound port opens.

The plan was to run cloudflared on the VM. Three routes were tested. All three failed, for hard technical reasons:

1. **Quick Tunnel.** Cloudflare's instant test tunnel service refused this machine. Provisioning returned HTTP 403.
2. **QUIC protocol.** A named tunnel prefers QUIC, which rides on UDP traffic. This VM blocks UDP egress. The connection attempt failed with "operation not permitted".
3. **HTTP2 protocol.** The fall-back protocol failed at the TLS step. This VM's DNS is intercepted (fake-IP), so hostnames resolve to a local pool of addresses. The edge server then presented a certificate for a different name than the client expected, and the secure connection could not form. Pinning a real edge address by number also failed: the edge closed the connection.

Each failure was verified with direct tests, not assumed. Lesson: when a machine's network is managed by someone else, some roads are simply closed. The fix is not to push harder on a closed road. The fix is to move the part that needs the open road.

## (D) THE PC-BRIDGE ARCHITECTURE

Here is the move that made the whole build work. The tunnel connector does not care where it runs. It only needs to reach two things: the Cloudflare edge, and the n8n server. Jesun's home PC has normal internet. n8n stays on the VM. So the connector moved to the PC, and a private bridge links the PC back to the VM.

The pieces:

1. **cloudflared on the PC.** Version 2026.10.0, installed on BlackBox. It holds a named tunnel, "jesun-n8n". On start, it registered 4 tunnel connections to Cloudflare's Toronto (yyz) edge over QUIC. The home network allows UDP, so QUIC works there.
2. **Tailscale link.** The PC and the VM sit on one private Tailscale network. The PC's address on that network is 100.100.217.73. SSH (secure shell) runs over this link. No public SSH port exists anywhere.
3. **SSH reverse forward.** The VM opens an SSH session to the PC. Inside that session, a reverse forward binds port 15678 on the PC's own loopback. Anything that connects to the PC's 127.0.0.1:15678 rides back through the SSH session and lands on the VM's

127.0.0.1:5678, where n8n listens.

4. **Tunnel ingress rule.** The PC's cloudflared config says: traffic for n8n.jesunahmadushno.com goes to http://localhost:15678. So the full path is: visitor, Cloudflare edge, tunnel to PC, port 15678, SSH forward, VM port 5678, n8n.
5. **The respawn loop.** One flaw showed up fast: SSH sessions through the runtime proxy die with "Broken pipe" after about a minute of quiet. A small loop script on the VM watches the forward. When the SSH session exits, the loop starts a new one. The loop runs detached, so it survives terminal closes.

The end-to-end proof ran from the PC itself: a call to the bridge port's health address returned the n8n OK status. Later, Cloudflare's own records showed the tunnel connector live from the PC, joined to Toronto edge nodes.

## (E) MOVE THE DOMAIN TO CLOUDFLARE WITHOUT BREAKING EMAIL OR THE WEBSITE

A tunnel route needs the domain's DNS on Cloudflare. The domain jesunahmadushno.com lived at Namecheap, with DNS that mattered:

- The website runs on GitHub Pages (4 A records at the apex, plus a www CNAME).
- Email forwarding runs on 5 MX records that point to the registrar's forwarders.
- One SPF TXT record supports the email setup.

Breaking any of these would break the site or the mail. So the move went in a safe order:

1. Add the zone to Cloudflare on the Free plan.
2. Check Cloudflare's automatic DNS scan against the public truth. The scan found exactly 11 records, and they matched the live records one by one. Site records were set to Proxied. Mail records stayed DNS-only, as mail records must.
3. Create the tunnel route inside Cloudflare: a record for n8n.jesunahmadushno.com that points at the tunnel.
4. Only then change the nameservers at Namecheap to the two Cloudflare names: autumn.ns.cloudflare.com and kenneth.ns.cloudflare.com.
5. Verify after the switch: the public NS lookup showed the two Cloudflare names, the zone flipped to Active, the apex resolved to Cloudflare addresses, all 5 MX records still answered, the website loaded as before, and the n8n address resolved through Cloudflare.

The zone was created before the nameserver change. That order is the whole trick: the new DNS host is fully loaded before the world is told to ask it.

## (F) THE ZERO TRUST WALL IN FRONT OF THE EDITOR

A public automation editor is a target. Anyone who reaches the login page can try passwords. Anyone who reaches an owner setup page first could claim the instance. So the editor got a

wall in front of it: Cloudflare Zero Trust Access.

Setup:

1. Activate Zero Trust on the Free plan. Activation asked for a payment method on file, with $0 due. The operator added his own card himself. Nothing was charged.
2. Create one self-hosted Access application. Name: "n8n Editor". Domain: n8n.jesunahmadushno.com.
3. Add one policy. Name: "Jesun only". Action: Allow. Rule: Emails, and one address only, the operator's own Gmail address.
4. Result: a visitor who is not on that one-address list never reaches n8n at all. Cloudflare stops them at the Access sign-in page. The operator signs in (the page offers the Cloudflare sign-in, backed by his account session) and passes through to the editor.

Note for later phases: webhook paths will need their own rule when webhook workflows go live (a bypass or a service token for those paths only). The editor itself stays behind the wall.

## (G) OWNER SETUP AND VERIFICATION

The last step of the build belonged to the operator alone:

1. Jesun opened the public address, passed the Access sign-in, and reached the n8n owner setup form.
2. He set the owner account himself: his email, his name, and a password he chose. Passwords are never invented by the agent and never stored in notes.
3. Verification, in the editor: the workflows were listed, and all 6 Google credentials sat in the store with the right names and types.
4. Cleanup followed the build: 4 test workflows from the acceptance tests were deleted (this n8n build deletes a workflow only after you archive it), the leftover unused OAuth client secret row on the Google client was deleted, and only the live secret row remains.

## (H) THE HONESTY CHAPTER: WHAT BROKE, AND WHAT IS STILL WEAK

A tutorial that hides its failures teaches nothing. Here is what went wrong in one night, on the record.

**The machine restarted 6 times in one night.** The cloud machine was replaced or rotated again and again (confirmed: each boot started from a fresh system state; the reason is platform-side and unverified). Every restart killed n8n and the bridge loop. A self-built guard, the "system reset guardian", watched for this: a hook checks the machine's boot ID every 60 seconds, and an hourly audit job backs it up. Across the night, the guardian detected each reset in 47 to 78 seconds. Work was restored from checkpoints each time. Total work lost across all 6 resets: zero.

**Error 1033.** At one point the public address showed Cloudflare error 1033: the tunnel had no live connector. Cause: the cloudflared process on the PC had been started inside an SSH session, and it died when that session closed. Fix: start the process detached from any session, and add a Windows scheduled task ("CloudflaredTunnel", on logon) so the connector starts again after a PC reboot.

**The editor showed offline.** Once, the editor reported it could not connect, and the browser showed a DNS error (NXDOMAIN) for the subdomain. Server side, every check was green at that moment: zone Active, tunnel Healthy, records present. The fault sat in the browser's own resolver cache during propagation. It cleared as DNS propagated.

**Named weak points (still true now):**

- The home PC is part of the chain. If the PC sleeps or is off, the public address sleeps with it.
- PC reboot is covered by the scheduled task. A sleeping PC is not covered.
- VM rotations still restart the services. n8n and the bridge loop need their restart steps after each rotation. The guardian detects and reports; the restore is scripted.
- This is a personal automation stack. It is not hosting for other people, and no one should copy it as one.

## (I) WHAT COMES NEXT: PHASE 2 HAS STARTED

Connection work continues in phases. Phase 2 is Meta: Facebook Page, Instagram, Threads, and Reddit reads.

Done so far, the same night:

- A Meta developer app in Jesun's own name: "Jesun n8n Automation" (App ID 2385437782261065, a public identifier).
- Two products added to the app: Instagram, and Facebook Login for Business.
- The n8n OAuth redirect address is registered in the app, ready for the wiring step.
- The app's Basic settings carry the domain and a privacy policy address (the operator's homepage for now; a full privacy page is owed and drafted).

Not done, stated plain: no Meta service is connected yet. Wiring, tokens, and live tests are pending. YouTube upload is also still untested (see step (b)). Claims stop where the tests stop. That is the rule of this series.

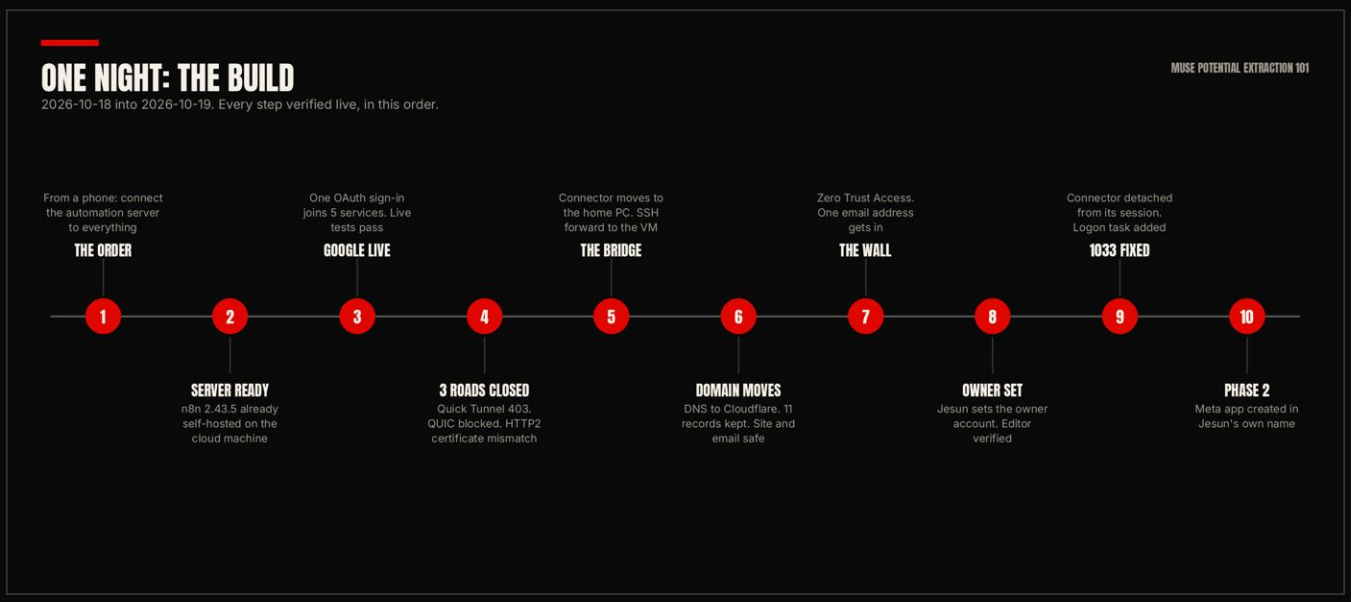



Figure 2. The build night, step by step, 2026-10-18 into 2026-10-19.

## TROUBLESHOOTING TABLE

| Symptom | Cause | Fix |
|---|---|---|
| Quick Tunnel will not provision (HTTP 403) | Cloudflare's instant tunnel service refuses this VM | Stop using Quick Tunnel on the VM. Use a named tunnel from a machine with open egress (the PC bridge) |
| Named tunnel fails on QUIC, "operation not permitted" | The VM blocks UDP egress | Run the connector where UDP is allowed, or fall back to HTTP2 where the network allows it |
| HTTP2 tunnel fails the TLS handshake | Sandbox DNS interception (fake-IP) makes the edge certificate name mismatch the expected name | Move the connector off the VM. The PC bridge removes the clash |
| SSH forward dies with "Broken pipe" after about a minute | The runtime proxy drops quiet SSH sessions | Run a respawn loop on the VM that restarts the forward on exit, with keep-alive traffic |
| Cloudflare error 1033 on the public address | The connector process on the PC died, here because it was tied to a closed SSH session | Start cloudflared detached from any session. Add a logon scheduled task so it returns after reboot |
| n8n says "already running", but the health check is empty | A stale pid file survives a crash or rotation, and the start script trusts it | Remove the stale pid file, then run the start script again |
| Browser shows NXDOMAIN for the new subdomain, while public DNS checks pass | The browser's resolver still holds the old delegation during propagation | Wait for propagation. Verify with a public resolver (DNS over HTTPS) instead of trusting one browser |

# THE $0 COST TABLE

| Item | Plan | Cost |
|---|---|---|
| Cloudflare DNS zone for jesunahmadushno.com | Free plan | $0 |
| Cloudflare Tunnel (jesun-n8n) | Included with the zone | $0 |
| Cloudflare Zero Trust Access ("n8n Editor" app) | Zero Trust Free (a card sits on file; $0 was due) | $0 |
| Google Cloud project jesun-n8n (5 APIs, one OAuth client) | No billing account created | $0 |
| n8n 2.43.5 | Self-hosted, open source | $0 |
| **Total spend for the build** | Free tiers only | **$0** |

The standing rule behind this table: free resources and free plans only. Any step that needs spend gets flagged and re-routed to a free path, or parked for the operator's explicit call.

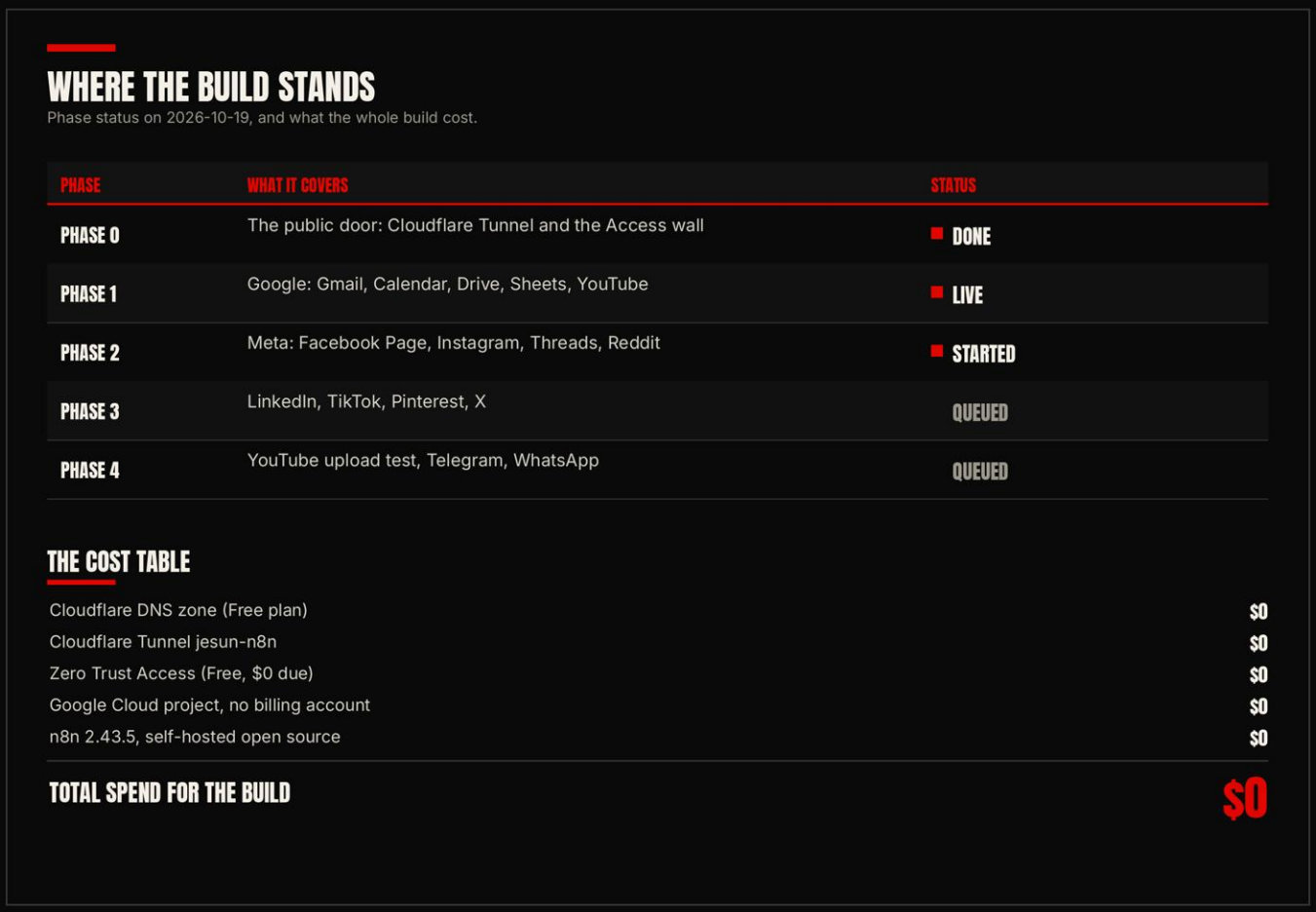



Figure 3. Phase status and the cost table. Free tiers only.

# HOW TO COPY THIS PATTERN

If you run an agent with its own cloud machine, the transferable lessons are these:

1. Keep the server loopback-only until you choose its door. Then add exactly one door.
2. Connect the big platform families first. One Google OAuth credential covered 5 services in one consent.
3. Test at the API level, then at the node level. A stored credential is not a connection.
4. When the machine's network blocks the standard route, move the connector, not the server. A home PC with normal internet is a fine bridge when a private network links them.
5. Guard the door before you open it. Access policy first, public test second.
6. Write down what broke. The failure log is the most reused page of this whole build.

## CLAIMS NOTE

Facts used from the production fact sheet (all live-verified 2026-10-18/19 unless noted): n8n version 2.43.5, local install, SQLite database, 8 starter workflows; Google Cloud project jesun-n8n; 5 Google services connected with one OAuth consent (Gmail, Calendar, Drive, Sheets, YouTube); all 5 passed live API tests with real data; YouTube channel "Jesun Ahmad Ushno" with 69 subscribers at the time of the test; 3 services also passed node-level tests; 6 Google credentials in the n8n store; the 3 failed tunnel routes on the VM (quick tunnel 403, QUIC/UDP blocked, HTTP2 certificate name mismatch through sandbox DNS); PC bridge architecture (cloudflared 2026.10.0 on BlackBox, Intel Core Ultra 7 255HX, 15 GB RAM, RTX 5050; 4 QUIC connections to Toronto yyz edge; PC tailnet address 100.100.217.73; SSH reverse forward, PC port 15678 to VM port 5678; respawn loop for dropped SSH sessions); tunnel name jesun-n8n; 11 DNS records preserved (GitHub Pages site, 5 MX records, SPF); nameservers autumn.ns.cloudflare.com and kenneth.ns.cloudflare.com; Zero Trust Free with Access application "n8n Editor" and a single Allow policy for the operator's email only; owner account set by Jesun through the public editor; 6 VM resets in one night with 0 work lost; guardian detection in 47 to 78 seconds (boot ID check every 60 seconds, hourly audit); error 1033 cause and fix (detached process, logon scheduled task); Meta Phase 2 started (app "Jesun n8n Automation", App ID 2385437782261065, Instagram and Facebook Login for Business products added, OAuth redirect registered, not yet connected); YouTube upload untested; total spend $0 on free tiers. The reset cause is stated as unverified, as in the fact sheet. No prices, speeds, or benchmarks beyond the fact sheet were used.

# SUPPORTING PACK

The one-page step checklist. The failure and fix table. A glossary of 12 terms. The TUNNEL comment trigger, explained.

Companion to the full tutorial. This pack is the quick-reference set: build the same setup from the checklist, fix the known failures from the table, and look up any term in the glossary.

## ONE-PAGE STEP CHECKLIST

### Stage 1: The server

- ☐ Install n8n on your machine (here: local install, version 2.43.5, SQLite database).
- ☐ Bind the editor to loopback only (127.0.0.1, port 5678). No public door yet.
- ☐ Confirm the health address answers OK.
- ☐ Leave all starter workflows switched off until you mean to run them.

### Stage 2: Google, one credential, 5 services

- ☐ Create one Google Cloud project. Skip billing.
- ☐ Enable the APIs: Gmail, Calendar, Drive, Sheets, YouTube Data API v3.
- ☐ Set the OAuth consent screen to "In production" (this kills the 7-day refresh token trap).
- ☐ Create one Web OAuth client. Register the n8n callback address.
- ☐ If consent fails on scope clash, drop narrow scopes that a wider scope already covers, and retry.
- ☐ Store the client ID and secret in n8n only. Never in files or notes.
- ☐ Make one typed credential copy per service (Gmail, Calendar, Drive, Sheets, YouTube).
- ☐ Test at API level with real data. Then test at node level inside n8n. A stored credential is not a connection.

### Stage 3: The door (tunnel)

- ☐ First test if your server machine can run cloudflared itself. If its egress blocks you (403 on quick tunnels, UDP blocked, certificate mismatch), move the connector. Do not fight the network.
- ☐ Run cloudflared on a machine with normal internet (here: the home PC, version 2026.10.0). Create a named tunnel.
- ☐ Link that machine back to the server over a private network (here: Tailscale plus SSH).
- ☐ Set an SSH reverse forward: connector machine port 15678 to server port 5678.
- ☐ Wrap the forward in a respawn loop. Quiet SSH sessions die; the loop is not optional.
- ☐ Point the tunnel ingress at the forward port. Prove the chain with a health call.

### Stage 4: The domain

- ☐ Add the zone to Cloudflare (Free plan) before you touch nameservers.
- ☐ Compare the DNS scan against live records, one by one. Site records Proxied. Mail records DNS-only.

- ☐ Add the tunnel route record for your subdomain.
- ☐ Change nameservers at the registrar to the two Cloudflare names.
- ☐ Verify after the switch: NS lookup, zone Active, site loads, all MX records answer, subdomain resolves.

### Stage 5: The wall

- ☐ Activate Zero Trust (Free plan) before the address goes public in any post.
- ☐ Create one Access application for the editor subdomain.
- ☐ One Allow policy, one email address: yours.
- ☐ Public test: the Access sign-in page must appear for a signed-out visitor.

### Stage 6: Owner and cleanup

- ☐ The owner sets the n8n owner account in person, with a self-chosen password.
- ☐ Verify in the editor: workflows listed, credentials listed with right names and types.
- ☐ Delete test workflows (archive first, then delete) and delete unused OAuth secret rows.

## FAILURE AND FIX TABLE

| # | Failure (what you see) | Cause | Fix |
|---|---|---|---|
| 1 | Quick Tunnel 403 | Cloudflare's instant tunnel provisioning refuses the VM | Drop Quick Tunnel on that machine. Use a named tunnel from a machine with open egress |
| 2 | QUIC tunnel blocked ("operation not permitted") | The VM blocks UDP egress, and QUIC rides on UDP | Run the connector where UDP is allowed (the home PC). The VM is the wrong host for the connector |
| 3 | HTTP2 tunnel fails on certificate name | Sandbox DNS interception (fake-IP) breaks the expected edge certificate name | Move the connector off the intercepted network. Pinning edge IPs also failed here; relocation is the fix |
| 4 | SSH broken pipe (forward dies after about a minute) | The runtime proxy drops quiet SSH sessions | Run a detached respawn loop that restarts the forward on exit, with keep-alive traffic |
| 5 | Cloudflare error 1033 | The connector process died; here it was bound to a closed SSH session, so the tunnel had no live connector | Start cloudflared detached from any session. Add a Windows logon scheduled task so it returns after reboot |

| # | Failure (what you see) | Cause | Fix |
|---|---|---|---|
| 6 | Stale pid file ("already running", but nothing answers) | A crash or rotation left the pid file behind, and the start script trusts it | Delete the pid file, then start again. Check the health address before you believe any status line |
| 7 | DNS propagation NXDOMAIN in one browser | That resolver still holds the old nameserver delegation while the change spreads | Verify with a public resolver (DNS over HTTPS). Wait for propagation. Do not rebuild a healthy server |

## GLOSSARY (12 TERMS)

1. **Sandbox (VM):** The agent's own cloud computer. A Linux virtual machine with a terminal, files, and a home folder that persists across restarts.
2. **n8n:** Open source workflow automation software. You connect services in visual workflows so tasks run on their own.
3. **Workflow:** One automation inside n8n: a chain of steps (a trigger, then actions) that runs as a unit.
4. **Credential:** The stored login material (tokens, client ID and secret) that lets a workflow act on an account. In n8n it sits in an encrypted store.
5. **OAuth:** The standard sign-in grant flow. You approve an app on the provider's own page; the app gets tokens; your password never goes to the app.
6. **Cloudflare Tunnel (cloudflared):** A program that dials out from your machine to Cloudflare and holds the link open, so visitors reach your server with no inbound port open.
7. **Reverse forward:** An SSH trick: a port on machine A is bound so that connections to it travel through the SSH session and land on a port on machine B.
8. **Tailscale:** A private network that links your own devices with private addresses, over the public internet, without public ports.
9. **DNS and nameservers:** DNS is the directory that turns a domain name into addresses and records. Nameservers are the hosts that hold a domain's directory; moving them moves the domain's control.
10. **Zero Trust Access:** Cloudflare's gate in front of an app. It checks who you are against a policy before the app itself ever sees your request.
11. **Webhook:** A call one service makes to another service's address when an event happens. It needs a public address to land on.
12. **Respawn loop:** A small watcher script that restarts a process every time it exits, so a fragile connection stays up without a human watching it.

## COMMENT TRIGGER EXPLAINER: KEYWORD TUNNEL

This campaign gives the full tutorial pack away through comments, on purpose.

**How it works for the reader:** comment the word **TUNNEL** on the campaign post. The pack (this supporting pack, the checklist, the tables, the glossary) is then delivered by DM. No link hunting, no landing page form.

**Why comments and not a link:** on Facebook, the standing rule for this brand is no link in the post body and no link in the comments. A keyword comment starts a real conversation, the DM carries the pack, and the post itself stays clean. The same trigger runs on the other platforms with each platform's own link rule kept where a link is allowed at all: on LinkedIn the link goes in the first comment, on Instagram the route is "link in bio", on Threads the link sits in the post body (under 500 characters), on X the link goes in the first reply.

**How it works behind the scenes:** the keyword is the trigger, a human-grade reply confirms it, and fulfilment is by DM. The pack is a fixed set of files, so every reader gets the same verified documents. If the keyword changes in a later campaign, the pack text is reissued; this edition's keyword is TUNNEL.

## CLAIMS NOTE

Facts used from the production fact sheet (all live-verified 2026-10-18/19): n8n 2.43.5 local install with SQLite, loopback port 5678; Google Cloud project jesun-n8n without billing; consent screen "In production" and the 7-day token trap; the scope clash error and its fix; 6 Google credentials (one base, 5 typed copies); API tests passed with real data for all 5 services, node-level tests passed for 3; the 3 failed VM tunnel routes (quick tunnel 403, QUIC/UDP blocked, HTTP2 certificate name mismatch through fake-IP DNS); the PC bridge (cloudflared 2026.10.0 on BlackBox, 4 QUIC connections to Toronto edge, Tailscale, PC tailnet address 100.100.217.73, SSH reverse forward 15678 to 5678, respawn loop after broken-pipe drops at about a minute); 11 DNS records preserved with site records Proxied and MX records DNS-only; nameservers autumn.ns.cloudflare.com and kenneth.ns.cloudflare.com; Zero Trust Free Access app "n8n Editor" with the single-email "Jesun only" policy; owner account set by Jesun in person; cleanup by archive-then-delete and removal of the unused secret row; error 1033 cause and fix (detached process plus logon scheduled task "CloudflaredTunnel"); stale pid file behaviour of the start script; the browser NXDOMAIN during propagation while public resolvers passed; the TUNNEL comment trigger with DM fulfilment and the per-platform link rules as recorded in the fact sheet. No facts beyond the fact sheet were used; the stale pid file and NXDOMAIN items follow the session record summarised in the production brief and the fact sheet's honesty chapter.